

3.2 Класифікація

У цьому розділі ми переходимо від прогнозування чисел до класифікації об'єктів. Розглядаються основні алгоритми класифікації, принципи побудови меж прийняття рішень та методи оцінювання якості моделей.

Перед початком роботи з алгоритмами класифікації необхідно підключити бібліотеки,

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay, RocCurveDisplay
from sklearn.preprocessing import StandardScaler
```

які відповідають за обчислення, підготовку даних, побудову моделей класифікації та оцінку їх якості:

- **make_classification** - генератор штучних даних для задач класифікації. Дозволяє швидко створити набір ознак та міток класів;
- **LogisticRegression** - реалізація алгоритму логістичної регресії для бінарної та мультикласової класифікації;
- **SVC (Support Vector Classifier)** - реалізація методу опорних векторів (SVM), який шукає оптимальну межу між класами;
- **classification_report** - інструмент для виведення основних метрик класифікації: precision, recall, f1-score та accuracy;
- **confusion_matrix** - створює матрицю помилок класифікації;
- **ConfusionMatrixDisplay** - використовується для графічного відображення confusion matrix;
- **RocCurveDisplay** - використовується для побудови ROC-кривої (Receiver Operating Characteristic).

3.2.1 Логістична регресія

У задачах класифікації модель повинна визначити, до якого класу належить об'єкт. Тобто, якщо регресія повертає дійсне число, то класифікація прогнозує категорію.

```
X, y = make_classification(n_samples=500, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Тут `make_classification()` містить: `n_samples=500` - 500 об'єктів, `n_features=2` - кожен об'єкт містить дві ознаки, `n_redundant=0` - немає лінійно залежних ознак, `n_clusters_per_class=1` - кожен клас створює один кластер точок.

Основою логістичної регресії є сигмоїдна функція (sigmoid function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Функція приймає будь-яке число та повертає значення від 0 до 1. Наприклад 0.95 - модель майже впевнена у класі 1, 0.10 - ймовірніше клас 0.

3.2.2 Метод опорних векторів (SVM)

Метод опорних векторів (Support Vector Machine, SVM) шукає гіперплощину, яка максимально добре розділяє класи. Головна мета - не просто повернути межу, а зробити максимальний зовнішній відступ (margin) між класами.

```
svm_model = SVC(kernel='linear', C=1.0, probability=True)
svm_model.fit(X_train_scaled, y_train)

print("--- SVM Results ---")
y_pred_svm = svm_model.predict(X_test_scaled)
print(classification_report(y_test, y_pred_svm))
```

Тут `SVC()` створює модель методу опорних векторів: `kernel='linear'` - використовується лінійна межа розділення; `C=1.0` - параметр регуляризації; `probability=True` - дозволяє моделі оцінювати ймовірність класів.

Під час навчання *SVM* намагається знайти оптимальну межу між класами з максимальним зовнішнім відступом. Зовнішній відступ (*margin*) - це відстань між межею рішень (*decision boundary*) та найближчими точками:

$$\text{Margin} = \frac{2}{\|w\|^1}$$

Чим більший зовнішній відступ, тим стабільнішою зазвичай є модель на нових даних.

Після навчання використовується:

```
y_pred_svm = svm_model.predict(X_test_scaled)
```

Метод `predict()` - прогнозує класи для тестової вибірки.

Опорні вектори (*support vectors*) - це точки, які знаходяться найближче до межі. Саме вони визначають положення межі рішень. Якщо видалити інші точки, то межа майже не зміниться.

Параметр *C* контролює баланс між шириною *margin* та кількістю помилок класифікації. Чим більше *C* - тим менше помилок допускає модель, але більший ризик перенавчання.

Якщо дані не можна розділити прямою лінією, використовується *kernel trick*. *Kernel* переводить дані у простір вищої розмірності, де вони вже можуть бути лінійно роздільними.

У *SVM* доступні різні типи *kernel*: `linear` - лінійна межа; `rbf` - нелінійна межа; `poly` - поліноміальна межа.

3.2.3 Метрики оцінки. Матриця помилок та ROC-крива

Після підготовки даних та масштабування можна перейти до навчання моделі логістичної регресії.

¹ $\|w\|$ - довжина (норма) вектора.

```
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)

print("--- Logistic Regression Results ---")
y_pred_log = log_reg.predict(X_test_scaled)
print(classification_report(y_test, y_pred_log))
```

Матриця помилок (Confusion Matrix) - таблиця помилок класифікації, містить: **TP (True Positive)** - правильне визначення позитивного класу; **TN (True Negative)** - правильне визначення негативного класу; **FP (False Positive)** - клас хибно визначено позитивним; **FN (False Negative)** - клас хибно визначено негативним.

Загальна точність (Accuracy) показує частку правильних відповідей.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Але якщо дані не збалансовані (один клас зустрічається набагато частіше за інший), загальна точність може бути оманливою. Наприклад, якщо 95% листів - це звичайні повідомлення, а лише 5% - спам, модель може завжди відповідати «не спам» і все одно отримати accuracy 95%. Хоча насправді така модель нічому не навчилась.

Точність (Precision) - показує, скільки позитивних прогнозів були правильними.

$$Precision = \frac{TP}{TP + FP}$$

Повнота (Recall) - показує, скільки реальних позитивних об'єктів знайшла модель.

$$Recall = \frac{TP}{TP + FN}$$

Тобто, припустимо є 100 листів, 10 з них - спам. Модель позначила як спам 8 листів, але з цього її списку лише 5 виявилися дійсно спамом. Тоді:

- $Precision = \frac{5}{8}$ - наскільки можна довіряти позитивним прогнозам моделі?
- $Recall = \frac{5}{10}$ - яку частину всього спаму модель змогла знайти?

F1-score поєднує precision та recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Ця метрика особливо корисна для незбалансованих даних.

Матриця помилок та ROC-крива

Для детального аналізу роботи класифікатора використовується confusion matrix - матриця помилок класифікації.

```
print("--- Візуалізація помилок (Confusion Matrix) ---")

fig, axes = plt.subplots(1, 2, figsize=(12, 6))

ConfusionMatrixDisplay.from_estimator(svm_model, X_test_scaled, y_test,
                                     cmap='Blues', ax=axes[0])
axes[0].set_title("Матриця помилок")

RocCurveDisplay.from_estimator(svm_model, X_test_scaled,
                               y_test, curve_kwargs={'color': 'blue'}, ax=axes[1])
axes[1].set_title("ROC-крива")

plt.tight_layout()
plt.show()
```

`ConfusionMatrixDisplay.from_estimator()` автоматично виконує прогнозування, обчислює матрицю помилок та будує її графічне відображення.

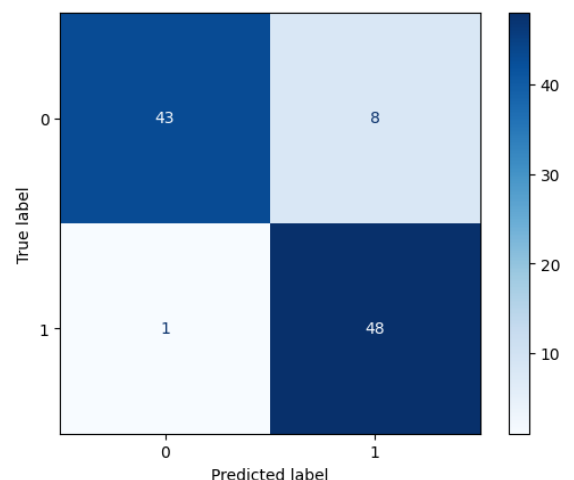
Тут: `svm_model` - навчена модель класифікації; `X_test_scaled` - тестові ознаки; `y_test` - правильні мітки класів; `cmap='Blues'` - кольорова схема візуалізації.

`RocCurveDisplay.from_estimator()` отримує ймовірності класів, обчислює ROC-криву та будує графік залежності TPR від FPR.

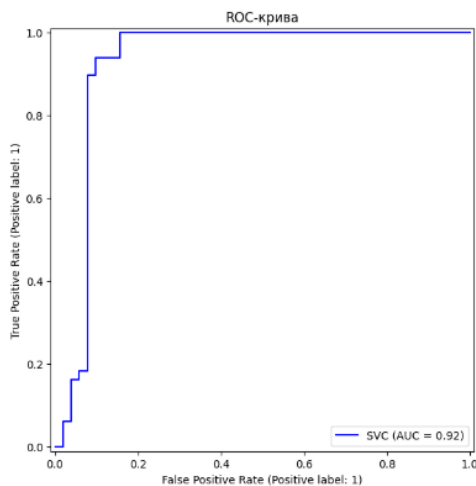
Після цього `plt.show()` відображає графік на екрані.

Зазвичай матриця помилок виглядає так:
 $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$. Вона дозволяє оцінити кількість правильних прогнозів, побачити типи помилок моделі та визначити, який клас модель плутає найчастіше.

На відміну від звичайного асигасу, матриця помилок показує повну картину роботи класифікатора. Тому вона особливо важлива для незбалансованих даних.



Наприклад, у медичній діагностиці FN-помилки можуть бути критичними, оскільки модель пропускає хворобу. Тому матриця помилок допомагає оцінити не лише кількість помилок, а й їх тип та критичність.



ROC (Receiver Operating Characteristic) – графік, який показує якість класифікації при різних порогах прийняття рішення.

По осі X показується FPR (False Positive Rate) – частка класів, хибно визначених позитивними. По осі Y показується TPR (True Positive Rate) – частка правильно знайдених позитивних об'єктів.

Формули:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

Ідеальна модель проходить максимально близько до верхнього лівого кута графіка, тобто показує високий TPR та низький FPR. Якщо крива знаходиться близько до діагоналі – модель працює майже випадково.

ROC-крива корисна для порівняння кількох моделей.

Основні кольори для матриці помилок

Відтінкові (одноколірні) схеми			
Blues	синій	Reds	червоний
Greens	зелений	Purples	фіолетовий
Oranges	помаранчевий	Greys	сірий
Перехідні (градієнтні) схеми			
viridis	фіолетовий - зелений	plasma	фіолетовий - жовтий
inferno	чорний - червоний - жовтий	magma	темний - рожево- фіолетовий
cividis	жовто-зелений		

3.2.4 Межа рішень (Decision Boundary)

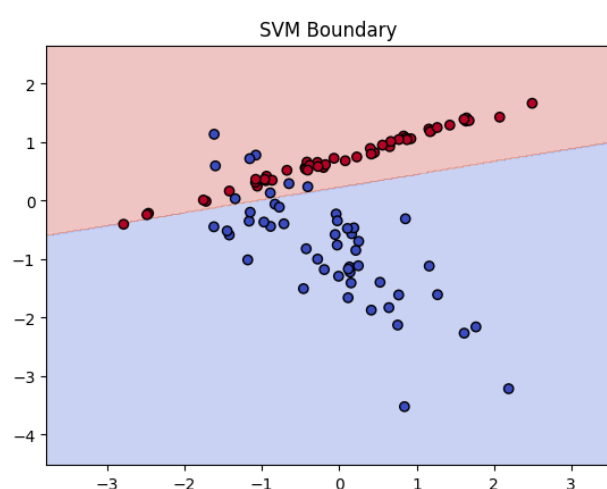
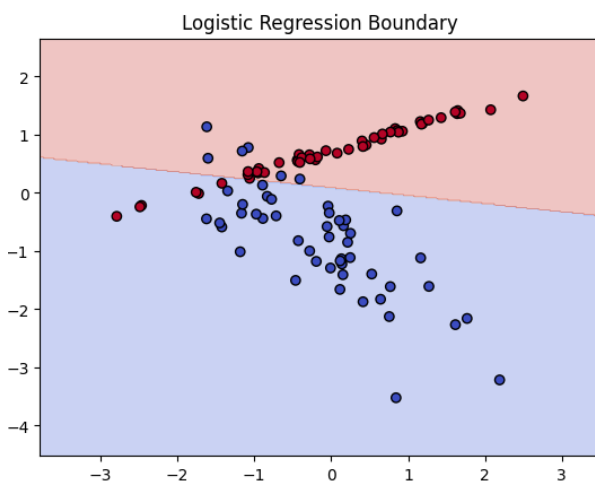
Після обчислення ймовірності (річ за сигмоїдну функцію, ймовірність належності до класу) модель формує Decision Boundary - межу прийняття рішень, яка розділяє класи у просторі ознак. Для логістичної регресії вона зазвичай є прямою лінією:

$$w_1x_1 + w_2x_2 + b = 0$$

Модель намагається знайти таку межу, щоб максимально добре розділити об'єкти різних класів.

Візуальне порівняння логістичної регресії та SVM:

```
def plot_decision_boundary(model, X, y, title):  
    h = .02  
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1  
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1  
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max,  
h))  
  
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])  
    Z = Z.reshape(xx.shape)  
  
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)  
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)  
    plt.title(title)  
    plt.show()  
  
plot_decision_boundary(log_reg, X_test_scaled, y_test, "Logistic Regression  
Boundary")  
plot_decision_boundary(svm_model, X_test_scaled, y_test, "SVM Boundary")
```



Функція `plot_decision_boundary()` створює сітку точок у просторі ознак та визначає, до якого класу модель відносить кожну точку.

Для цього: `meshgrid()` - створює координатну сітку; `predict()` - класифікує всі точки сітки; `contourf()` - зафарбовує області різних класів; `scatter()` - відображає реальні точки даних.

3.2.5 Наївний Баєсів (Naïve Bayes)

У цьому розділі розглядаються основи обробки природної мови (Natural Language Processing, NLP) та аналіз тональності тексту. Sentiment Analysis використовується для автоматичного визначення емоційного забарвлення тексту: позитивного, негативного або нейтрального.

Перед початком роботи необхідно підключити бібліотеки:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
```

- `TfidfVectorizer` - перетворює текст у числові ознаки за допомогою TF-IDF (Term Frequency - Inverse Document Frequency). Модель машинного навчання не може працювати з текстом напряду, тому слова переводяться у вектори чисел;
- `MultinomialNB` - реалізація наївного байєсівського класифікатора (Naive Bayes), який часто використовується для задач класифікації тексту;
- `Pipeline` - дозволяє об'єднати кілька етапів обробки даних (наприклад `vectorizer` + `classifier`) у єдину модель;

Обробка тексту та виділення ознак

Моделі машинного навчання не можуть працювати з текстом напряду. Перед навчанням текст необхідно перетворити у числовий формат, який можна аналізувати математично.


```
def load_data(pos_path, neg_path):  
    with open(pos_path, 'r', encoding='utf-8', errors='ignore') as f:  
        pos_text = f.read().splitlines()  
    with open(neg_path, 'r', encoding='utf-8', errors='ignore') as f:  
        neg_text = f.read().splitlines()  
  
    X = pos_text + neg_text  
    y = [1] * len(pos_text) + [0] * len(neg_text)  
    return train_test_split(X, y, test_size=0.2, random_state=42)  
  
X_train, X_test, y_train, y_test = load_data('rt-polarity.pos', 'rt-  
polarity.neg')  
print(f"Дані завантажено. Тренувальна вибірка: {len(X_train)} відгуків")
```

У цьому прикладі: `rt-polarity.pos` - містить позитивні відгуки; `rt-polarity.neg` - негативні відгуки. Для читання файлів використовується метод `open()`, тут: `encoding='utf-8'` - підтримка Unicode символів; `errors='ignore'` - ігнорування проблемних символів.

Після цього: `f.read().splitlines()` зчитує увесь текст та розбиває його на окремі рядки (окремі відгуки).

Після завантаження створюються: `X` - список текстів (об'єднує позитивні та негативні відгуки); `y` - створює мітки класів (1 - позитивні, 0 - негативні).

Перш ніж модель почне вчитися, текст проходить через кілька етапів обробки:

- **Токенізація (Tokenization)** - це розбиття тексту на окремі слова (токени). Вони стають базовими елементами для аналізу тексту.
- **Очищення (Preprocessing)** - виконується перед перетворенням тексту в числові ознаки: переводить текст в нижній регістр, видаляє пунктуацію стоп-слова (stop-words). Це допомагає зменшити шум, покращити якість моделі та зменшити кількість непотрібних ознак.
- **Стоп-слова (stop-words)** - це слова, що часто зустрічаються, але майже не несуть емоційного змісту («the», «and», «is», «a»).

Методи представлення тексту:

- **Bag of Words** - текст розглядається як набір слів, граматики та порядку слів ігноруються, враховується лише кількість появи слова. Такий підхід дозволяє перетворити текст у числовий вектор.

- Розумне зваження TF-IDF (Term Frequency - Inverse Document Frequency) - покращена версія Bag of Words, зменшує вагу дуже частих слів та збільшує вагу унікальних і важливих.

TF-IDF обчислюється:

$$TFIDF(t, d) = TF(t, d) \cdot \log \frac{N}{df(t)}$$

де:

- $TF(t, d)$ - частота слова у документі,
- $df(t)$ - кількість документів, де зустрічається слово,
- N - загальна кількість документів.

Саме TF-IDF найчастіше використовується у класичних NLP-моделях.

Класифікація тексту (Naïve Bayes)

Після перетворення тексту у числові ознаки можна переходити до навчання моделі класифікації. Одним із найпопулярніших алгоритмів для аналізу тексту є Наївний Байєс (Naïve Bayes).

```
text_clf = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', min_df=5, ngram_range=(1,
2))),
    ('clf', MultinomialNB())
])

text_clf.fit(X_train, y_train)
```

У цьому прикладі використовується Pipeline - спеціальний механізм sklearn, який дозволяє об'єднати кілька етапів обробки даних у єдиний процес. Тут pipeline складається з двох етапів: TfidfVectorizer - перетворення тексту у TF-IDF ознаки, MultinomialNB — класифікація тексту.

Аргументи TfidfVectorizer: stop_words='english', - автоматично видаляє англійські стоп-слова, min_df=5 - слово враховується тільки якщо воно зустрічається в 5 документах і більше, ngram_range=(1, 2) - модель аналізує окремі слова та пари слів, це допомагає їй краще зрозуміти контекст

Для класифікації використовується MultinomialNB() - це використання теореми Байєса у коді. У задачі аналізу тексту модель оцінює наскільки ймовірно, що текст позитивний, якщо в ньому зустрічаються певні слова. Тобто слово «excellent» збільшує ймовірність позитивного класу, слово «terrible» збільшує ймовірність негативного класу.

Алгоритм називається «наївним» через свій підхід до задачі. Модель вважає появу слово «good» ознакою, що клас ймовірно позитивний. Вона не враховує сарказм чи ситуацію того що людина просто привіталась (добрий день). Але для текстів результати часто виявляються хорошими, тому цей алгоритм ще активно використовується.

Оцінка якості моделі в NLP

Після навчання моделі необхідно перевірити, наскільки добре вона визначає тональність тексту на нових даних.

```
predictions = text_clf.predict(X_test)

print("\n--- Звіт про класифікацію ---")
print(classification_report(y_test, predictions, target_names=['Negative',
'Positive']))

sample_reviews = [
    "This movie was an absolute masterpiece with great acting!",
    "Waste of time. The plot was boring and predictable.",
    "Not bad, but I've seen better"
]

sample_preds = text_clf.predict(sample_reviews)
for review, pred in zip(sample_reviews, sample_preds):
    sentiment = "Positive" if pred == 1 else "Negative"
    print(f"Review: {review[:50]}... -> {sentiment}")
```

Метод: `predict()` використовується для прогнозування класів, тут: `X_test` - тестові відгуки, `predictions` - прогнозовані мітки класів, `target_names=['Negative', 'Positive']` - замінює числа 0 та 1 на зрозумілі назви класів.

Модель можна використовувати не лише для тестового набору, а й для довільних речень (оголомити змінну `sample_reviews`, після чого визначити її тональність `text_clf.predict(sample_reviews)`)

Також можна подивитись, які слова модель вважає найбільш позитивними (функція `show_top_words`).

```
def show_top_words(pipeline, n=10):  
    vectorizer = pipeline.named_steps['tfidf']  
    clf = pipeline.named_steps['clf']  
    feature_names = vectorizer.get_feature_names_out()  
  
    sorted_indices = np.argsort(clf.feature_log_prob_[1])[-n:]  
    print(f"\nТоп-{n} слів для позитивного класу:")  
    print([feature_names[i] for i in sorted_indices])  
  
show_top_words(text_clf)
```

Pipeline зберігає всі етапи моделі, а через `named_steps` можна отримати доступ до зваження тексту ('tfidf') та класифікатора ('clf'). Команда `np.argsort()` сортує слова за важливістю, тут: `feature_log_prob_` містить логарифми ймовірностей для кожного класу (тобто чим більше значення тим більше слово асоціюється з класом).